

Laboratorio #2 — Redes Neuronales Convolucionales (CNN vs. MLP en MNIST)

CC3092 Deep Learning y Sistemas Inteligentes

Ernesto Ascencio 23009

Repositorio: <https://github.com/AscencioSIUU/deepLearning/tree/main/labs/lab2-CNN>

1. Investigación de capas de PyTorch para CNN

`nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)` aplica convolución 2D con pesos compartidos: cada kernel se desliza sobre toda la imagen, conectando cada salida solo con una ventana local (campo receptivo), a diferencia de `nn.Linear` que conecta cada píxel con cada neurona. `nn.MaxPool2d(kernel_size)` reduce resolución tomando el máximo por ventana (sin parámetros); `nn.AvgPool2d` hace lo mismo con el promedio. `nn.BatchNorm2d(num_features)` normaliza activaciones por canal/mini-batch con reescalado aprendible, estabilizando el entrenamiento. `nn.Flatten` convierte `[batch,C,H,W]` en `[batch, C*H*W]` para conectar con capas `Linear`. `nn.CrossEntropyLoss` combina `log_softmax+NLLLoss`: espera **logits crudos** y etiquetas enteras, no probabilidades ni one-hot.

Tensor: arreglo multidimensional con autograd, ejecutable en CPU/GPU/MPS. **Campo receptivo:** región de la entrada que influye en una activación dada; crece al apilar capas conv/pool. **Por qué la CNN necesita menos parámetros:** weight sharing + conectividad local — un `Conv2d(1,8,kernel_size=3)` tiene 80 parámetros reutilizados en toda la imagen, frente a los ~100k de un `Linear(784,128)` que aprende un peso distinto por cada combinación píxel-neurona.

2. Datos y arquitecturas

MNIST: 60,000 train / 10,000 test, 10 clases balanceadas, imágenes 28×28 en escala de grises (`uint8 [0,255]`), normalizadas con `Normalize(0.1307, 0.3081)`. Split adicional train→train/val 90/10 (54k/6k); test intocable hasta la evaluación final. MLP: Flatten + capas `Linear` configurables. CNN: bloques `Conv2d+BatchNorm2d+activación+Pool` configurables + FC final. Mismo ciclo de entrenamiento (`train_model/run_epoch`) para ambas, con `nn.CrossEntropyLoss` y `torch.optim.Adam`.

3. Tabla de resultados (12 iteraciones: 6 MLP + 6 CNN, 6 épocas c/u)

#	Arq.	Descripción	Params	Val acc	Val loss	Tiempo
M1	MLP	[128,64], ReLU, Adam/1e-3 (baseline, mejor)	109,386	0.9732	0.0998	46s
M2	MLP	[256,128], ReLU, Adam/1e-3	235,146	0.9727	0.0921	35s
M3	MLP	[128,64], LeakyReLU	109,386	0.9708	0.1057	34s
M4	MLP	Adam/1e-2	109,386	0.9542	0.2297	35s

#	Arq.	Descripción	Params	Val acc	Val loss	Tiempo
M5	MLP	dropout=0.3	109,386	0.9667	0.1138	35s
M6	MLP	batch=256	109,386	0.9683	0.1055	21s
C1	CNN	conv=[16,32], BN, MaxPool (baseline)	105,962	0.9875	0.0430	74s
C2	CNN	conv=[32,64]	220,426	0.9873	0.0484	75s
C3	CNN	sin BatchNorm	105,866	0.9880	0.0420	71s
C4	CNN	AvgPool	105,962	0.9875	0.0454	73s
C5	CNN	dropout=0.3	105,962	0.9877	0.0435	74s
C6	CNN	weight_decay=1e-4 (mejor)	105,962	0.9883	0.0404	72s

Modelos finales (test, reentrenados 15 épocas, evaluados una única vez):

Arquitectura	Config	Params	Accuracy	Precision	Recall	F1	Tiempo
MLP	M1	109,386	0.9775	0.9774	0.9774	0.9774	90.5s
	baseline						
CNN	C6	105,962	0.9895	0.9894	0.9894	0.9894	188.3s
	weight_decay						

4. Comparación de arquitecturas

La CNN gana en accuracy de test (98.95% vs. 97.75%) usando **menos** parámetros entrenables (105,962 vs. 109,386) — es superior tanto en calidad como en tamaño del modelo. El costo está en tiempo de entrenamiento: ~2× más lento (188.3s vs. 90.5s en este equipo), porque cada kernel se aplica repetidamente en cada posición de la imagen (el ahorro de parámetros no implica menos cómputo por forward pass).

5. Discusión y análisis

Mayor impacto +/-: MLP — ningún cambio superó al baseline en 6 épocas; el mayor impacto negativo fue lr=1e-2 (M4: val_acc 0.9542, el peor de todos), un paso 10× mayor que sobre-corrige los pesos. CNN — weight_decay=1e-4 (C6) dio el mejor val_acc (0.9883), aunque el clúster CNN quedó muy apretado (0.9873-0.9883); ningún cambio tuvo impacto negativo notable.

Overfitting/underfitting: con 6 épocas la señal dominante fue underfitting leve (train y val bajan juntas); extender a 15 épocas en la evaluación final mejoró ambas arquitecturas (MLP 0.9732→0.9775, CNN 0.9883→0.9895 val→test), confirmando que el Batch 4 no había agotado la capacidad de mejora. No se observó overfitting agudo en ninguna configuración corta.

Regularización: en el MLP, dropout=0.3 empeoró (0.9667 vs. 0.9732 baseline) — restarle capacidad a un modelo que aún underfitteaba no ayudó. En la CNN, weight_decay sí mejoró levemente (0.9875→0.9883): un L2 suave penaliza pesos grandes sin quitar capacidad, funcionando mejor que dropout en un régimen sin overfitting.

MLP vs. CNN: la CNN ganó claramente (98.95% vs. 97.75%, F1 0.9894 vs. 0.9774) con menos parámetros. La diferencia viene de cómo procesan la información espacial: el MLP aplanar la imagen y pierde la vecindad entre píxeles; la CNN, con convolución de pesos compartidos y campo receptivo local, explota directamente que los trazos son patrones locales reutilizables en cualquier posición.

Errores más frecuentes (test, real→predicho: conteo): MLP: 8→3 (16), 6→5 (11), 9→4 (11); CNN: 2→7 (12), 9→4 (9), 6→5 (8). Ambos confunden pares de dígitos con trazos curvos similares (9↔4, 6↔5), pero

en pares distintos — el MLP falla más en 8→3 (pierde la relación espacial entre curva superior e inferior al aplanar); la CNN falla más en 2→7 (2s sin trazo curvo inferior). La CNN no elimina la ambigüedad de la escritura a mano, solo reduce su frecuencia.

Modelo de producción: CNN — gana en accuracy y F1, y lo hace con menos parámetros que el MLP, siendo superior en ambos ejes de calidad y tamaño de modelo. El único costo es tiempo de entrenamiento offline ($\sim 2\times$), no un costo recurrente por request; para un problema de visión como MNIST, modelar la estructura espacial es la elección correcta salvo un presupuesto de latencia de inferencia extremadamente estricto.